

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

**ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ № 5
«процедуры, функции, триггеры в PostgreSQL» по дисциплине
«Проектирование и реализация баз данных»
Вариант 2. БД «Сессия»**

Обучающийся Левчук Софья Александровна
Факультет прикладной информатики
Группа K3241
Направление подготовки 09.03.03 Прикладная информатика
Образовательная программа Мобильные и сетевые технологии
Преподаватель Говорова Марина Михайловна, Белов Александр Олегович

Санкт-Петербург
2025/2026

Создание процедур

1 Для повышения стипендии отличникам на 10%

Посмотрим на то, какие стипендии есть изначально:

```
UniversitySession=# SELECT scholarship_id, type, amount FROM scholarships;
```

scholarship_id	type	amount
1	Академическая	2300.00
2	Повышенная академическая	4300.00
3	Президентская	7000.00
4	Именная стипендия	6000.00
5	Спортивная	3000.00

Добавим новую стипендию с названием “Академическая(10%)” и ее назначим отличникам.

Выведем изначальные назначения стипендий:

```
UniversitySession=# SELECT sa.assignment_id, s.student_id, p.last_name, p.first_name, st.type, sa.is_active
UniversitySession=# FROM scholarship_assignments sa
UniversitySession=# JOIN students s ON sa.student_id = s.student_id
UniversitySession=# JOIN people p ON s.person_id = p.person_id
UniversitySession=# JOIN scholarships st ON sa.scholarship_id = st.scholarship_id
UniversitySession=# WHERE sa.is_active = true
UniversitySession=# ORDER BY s.student_id;
```

assignment_id	student_id	last_name	first_name	type	is_active
41	6	Беломытцев	Никита	Спортивная	t
6	6	Беломытцев	Никита	Президентская	t
42	7	Демидов	Владимир	Спортивная	t
7	7	Демидов	Владимир	Повышенная академическая	t
43	8	Петров	Цзясюань	Спортивная	t
8	8	Петров	Цзясюань	Президентская	t
44	9	Богачёв	Максим	Спортивная	t
9	9	Богачёв	Максим	Президентская	t
45	10	Мещанов	Даниил	Спортивная	t
46	11	Шестаков	Василий	Спортивная	t
47	12	Орешин	Михаил	Спортивная	t
12	12	Орешин	Михаил	Академическая	t
48	13	Шевченко	Данила	Спортивная	t
13	13	Шевченко	Данила	Именная стипендия	t
49	14	Ши	Никита	Спортивная	t
14	14	Ши	Никита	Повышенная академическая	t
50	15	Никитин	Эрик	Спортивная	t
15	15	Никитин	Эрик	Именная стипендия	t
51	16	Рахмани	Бяо	Спортивная	t
16	16	Рахмани	Бяо	Президентская	t
52	17	Дубинин	Николай	Спортивная	t
53	18	Каспер	Николай	Спортивная	t
54	19	Пань	Ханчжэ	Спортивная	t
55	20	Прохорова	Татьяна	Спортивная	t
20	20	Прохорова	Татьяна	Повышенная академическая	t
22	22	Чжоу	Семён	Повышенная академическая	t
23	23	Лемешко	Клим	Президентская	t
24	24	Васляев	Степан	Академическая	t
25	25	Яворский	Владимир	Именная стипендия	t
26	26	Попышева	Лена	Повышенная академическая	t
28	28	Дагаев	Дмитрий	Повышенная академическая	t
29	29	Ведерников	Даниял	Именная стипендия	t
30	30	Ян	Константин	Повышенная академическая	t
31	31	Митрик	Родион	Академическая	t
33	33	Чупрова	Ирина	Повышенная академическая	t
34	34	Люсин	Михаил	Президентская	t
35	35	Семёнов	Павел	Повышенная академическая	t
36	36	Ларионова	Ксения	Академическая	t
38	38	Семьнин	Алексей	Президентская	t
39	39	Чжоу	Яков	Президентская	t
40	40	Шаланов	Захарий	Повышенная академическая	t

Посмотрим на отличников:

```

UniversitySession=# SELECT s.student_id, p.last_name, p.first_name
UniversitySession=# FROM students s
UniversitySession=# JOIN people p ON s.person_id = p.person_id
UniversitySession=# WHERE s.status = 'Активен'
UniversitySession=# AND NOT EXISTS (
UniversitySession(# SELECT 1 FROM session_results sr
UniversitySession(# WHERE sr.student_id = s.student_id
UniversitySession(# AND sr.assessment_type = 'Экзамен'
UniversitySession(# AND sr.grade < 90
UniversitySession(# )
UniversitySession=# AND EXISTS (
UniversitySession(# SELECT 1 FROM session_results sr
UniversitySession(# WHERE sr.student_id = s.student_id AND sr.assessment_type = 'Экзамен'
UniversitySession(# );
student_id | last_name | first_name
-----+-----+-----

```

Таблица пуста, поэтому вручную сделаем одного из студентов отличником. Пусть это будет студент с id=3. Назначим всем экзамен балл 95, а зачетам - 70.

```

UniversitySession=# UPDATE session_results
UniversitySession=# SET grade = 95, status = 'Сдано'
UniversitySession=# WHERE student_id = 3 AND assessment_type = 'Экзамен';
UPDATE 8
UniversitySession=# UPDATE session_results
UniversitySession=# SET grade = 70, status = 'Сдано'
UniversitySession=# WHERE student_id = 3 AND assessment_type = 'Зачет';
UPDATE 4

```

Снова выведем таблицу отличников:

```

UniversitySession(# );
student_id | last_name | first_name
-----+-----+-----
          3 | Максудов | Михаил

```

Создадим нужную процедуру:

```

CREATE OR REPLACE PROCEDURE increase_scholarship()
LANGUAGE plpgsql AS $$

```

```

DECLARE

```

```

    excellent_stud RECORD;
    new_scholarship_id INT;
    base_amount NUMERIC;
    raised_amount NUMERIC;

```

```

BEGIN

```

```

    SELECT amount INTO base_amount FROM scholarships WHERE type = 'Академическая';
    IF NOT FOUND THEN

```

```

        RAISE EXCEPTION 'Не найдена стипендия "Академическая"';
    END IF;

```

```

    raised_amount := base_amount * 1.10;

```

```

    INSERT INTO scholarships (type, amount, conditions)

```

```

    VALUES ('Академическая(10%)', raised_amount,
            'Повышенная стипендия для отличников')

```

```

    ON CONFLICT (type) DO UPDATE SET amount = EXCLUDED.amount
    RETURNING scholarship_id INTO new_scholarship_id;

```

```

FOR excellent_stud IN
    SELECT s.student_id
    FROM students s

```

```

WHERE s.status = 'Активен'
AND NOT EXISTS (
    SELECT 1 FROM session_results sr
    WHERE sr.student_id = s.student_id
    AND sr.assessment_type = 'Экзамен'
    AND sr.grade < 90
)
AND NOT EXISTS (
    SELECT 1 FROM session_results sr
    WHERE sr.student_id = s.student_id
    AND sr.assessment_type = 'Зачет'
    AND sr.grade < 60
)
AND EXISTS (
    SELECT 1 FROM session_results sr
    WHERE sr.student_id = s.student_id AND sr.assessment_type = 'Экзамен'
)
LOOP
UPDATE scholarship_assignments
SET is_active = FALSE
WHERE student_id = excellent_stud.student_id
AND is_active = TRUE
AND scholarship_id IN (SELECT scholarship_id FROM scholarships
    WHERE type IN ('Академическая', 'Повышенная государственная
академическая'));

INSERT INTO scholarship_assignments (student_id, scholarship_id, assignment_date,
is_active)
VALUES (excellent_stud.student_id, new_scholarship_id, CURRENT_DATE, TRUE);
END LOOP;

RAISE NOTICE 'Отличникам назначена стипендия в размере %', raised_amount;
END;
$$;

```

Теперь вызовем процедуру и увидим сообщение о том, что отличникам назначена новая стипендия в определенном размере (тут я узнала о какой-то проблеме кодировки):

```

UniversitySession=# CALL increase_scholarship();
Р-РґРґР•Р$РґРґРР•: РґС,Р»РёС+Р$РёРёР°Рј Р$Р°Р•Р$Р°С+РґР$Р° СґС,РёРіРґР$РґРёСґ РІ СБР°Р•РјРґСБРґ 2530.0000
CALL

```

Видим, что новая стипендия действительно создана:

```

UniversitySession=# select * from scholarships;
scholarship_id | type | amount | conditions
-----
1 | Академическая | 2300.00 | Успеваемость без троек, сессия на "4" и "5"
2 | Повышенная академическая | 4300.00 | Только отличники (все оценки "5")
3 | Президентская | 7000.00 | Победители олимпиад, особые научные достижения
4 | Именная стипендия | 6000.00 | Участие в проектах, хакатонах, научных работах
ях
6 | Академическая(10%) | 2530.00 | Повышенная стипендия для отличников

```

Посмотрим на назначение стипендий для студента с id=3 и увидим назначение новой:

```
UniversitySession=# SELECT sa.student_id, st.type, sa.is_active, sa.assignment_date
UniversitySession=# FROM scholarship_assignments sa
UniversitySession=# JOIN scholarships st ON sa.scholarship_id = st.scholarship_id
UniversitySession=# WHERE sa.student_id = 3
UniversitySession=# ORDER BY sa.assignment_date DESC;
```

student_id	type	is_active	assignment_date
3	Академическая(10%)	t	2026-05-13

2 Для перевода студентов на следующий курс

Сейчас есть две группы (2 курс):

```
UniversitySession=# SELECT group_id, group_number, course, start_date, end_date
UniversitySession=# FROM student_groups
UniversitySession=# ORDER BY group_id;
```

group_id	group_number	course	start_date	end_date
1	K3241	2	2024-09-01	2028-06-30
2	K3240	2	2024-09-01	2028-06-30

Проверим, что в них есть активные студенты:

```
UniversitySession=# SELECT sg.group_id, sg.group_number, COUNT(s.student_id) AS active_students
UniversitySession=# FROM student_groups sg
UniversitySession=# LEFT JOIN students s ON s.group_id = sg.group_id AND s.status = 'Активен'
UniversitySession=# GROUP BY sg.group_id, sg.group_number
UniversitySession=# ORDER BY sg.group_id;
```

group_id	group_number	active_students
1	K3241	20
2	K3240	20

```
UniversitySession=# SELECT s.student_id, p.last_name, p.first_name, sg.group_number, sg.course
UniversitySession=# FROM students s
UniversitySession=# JOIN student_groups sg ON s.group_id = sg.group_id
UniversitySession=# JOIN people p ON s.person_id = p.person_id
UniversitySession=# WHERE s.status = 'Активен'
UniversitySession=# ORDER BY s.student_id
UniversitySession=# LIMIT 10;
```

student_id	last_name	first_name	group_number	course
1	Шнейдерис	Пётр	K3241	2
2	Гуань	Андрей	K3241	2
3	Максудов	Михаил	K3241	2
4	Яхина	Алексей	K3241	2
5	Хайкин	Елизавета	K3241	2
6	Беломятцев	Никита	K3241	2
7	Демидов	Владимир	K3241	2
8	Петров	Цзясюань	K3241	2
9	Богачёв	Максим	K3241	2
10	Мещанов	Даниил	K3241	2

(10 rows)

Код процедуры:

```
CREATE OR REPLACE PROCEDURE transfer_students_to_next_course()
LANGUAGE plpgsql AS $$
DECLARE
    old_group RECORD;
    new_group_id INT;
    new_group_number VARCHAR;
    new_course INT;
BEGIN
    FOR old_group IN
        SELECT sg.group_id, sg.group_number, sg.course, sg.start_date, sg.end_date
        FROM student_groups sg
        WHERE sg.course < 4
```



```
UniversitySession=# SELECT group_id, group_number, course, start_date, end_date
UniversitySession=# FROM student_groups
UniversitySession=# ORDER BY group_id;
```

group_id	group_number	course	start_date	end_date
1	K3241	2	2024-09-01	2028-06-30
2	K3240	2	2024-09-01	2028-06-30
3	K3341	3	2026-05-13	2027-05-13
4	K3340	3	2026-05-13	2027-05-13

```
UniversitySession=# SELECT s.student_id, p.last_name, p.first_name, sg.group_number, sg.course
UniversitySession=# FROM students s
UniversitySession=# JOIN student_groups sg ON s.group_id = sg.group_id
UniversitySession=# JOIN people p ON s.person_id = p.person_id
UniversitySession=# WHERE s.status = 'Активен'
UniversitySession=# ORDER BY s.student_id
UniversitySession=# LIMIT 10;
```

student_id	last_name	first_name	group_number	course
1	Шнейдерис	Пётр	K3341	3
2	Гуань	Андрей	K3341	3
3	Максудов	Михаил	K3341	3
4	Яхина	Алексей	K3341	3
5	Хайкин	Елизавета	K3341	3
6	Беломытцев	Никита	K3341	3
7	Демидов	Владимир	K3341	3
8	Петров	Цзясюань	K3341	3
9	Богачёв	Максим	K3341	3
10	Мещанов	Даниил	K3341	3

3 Для изменения оценки при успешной пересдаче экзамена

```
UniversitySession=# select * from session_results limit 5;
```

result_id	student_id	grade	assessment_type	attempt_number	exam_date	status	discipline_id	teacher_id
1	1	13	Зачет	1	2026-01-21	Не сдано	1	1
2	1	23	Зачет	1	2026-01-17	Не сдано	2	2
3	1	100	Экзамен	1	2026-01-02	Сдано	3	3
4	1	97	Экзамен	1	2026-01-23	Сдано	4	4
5	1	40	Экзамен	1	2026-01-16	Не сдано	5	5

Возьмем, например, результат сдачи экзамена по дисциплине с id=5 студента с id=1.

Код процедуры:

```
CREATE OR REPLACE PROCEDURE add_exam_retake(
    p_student_id INT,
    p_discipline_id INT,
    p_new_grade INT
)
LANGUAGE plpgsql AS $$
DECLARE
    v_max_attempt INT;
    v_new_attempt INT;
    v_has_success INT;
    v_assessment_type VARCHAR;
    v_teacher_id INT;
BEGIN
    IF p_new_grade < 0 OR p_new_grade > 100 THEN
        RAISE EXCEPTION 'Grade must be between 0 and 100';
    END IF;

    SELECT COUNT(*) INTO v_has_success
    FROM session_results
    WHERE student_id = p_student_id
    AND discipline_id = p_discipline_id
```



```

AND grade >= 60;

IF v_has_success > 0 THEN
    RAISE EXCEPTION 'Student already passed this discipline (grade >= 60)';
END IF;

SELECT COALESCE(MAX(attempt_number), 0) INTO v_max_attempt
FROM session_results
WHERE student_id = p_student_id AND discipline_id = p_discipline_id;

IF v_max_attempt = 0 THEN
    RAISE EXCEPTION 'No previous attempt found for this student and discipline (use
INSERT instead)';
END IF;

v_new_attempt := v_max_attempt + 1;

IF v_new_attempt > 3 THEN
    RAISE EXCEPTION 'Maximum 3 attempts allowed (current max = %)', v_max_attempt;
END IF;

SELECT assessment_type INTO v_assessment_type
FROM session_results
WHERE student_id = p_student_id AND discipline_id = p_discipline_id
ORDER BY attempt_number DESC LIMIT 1;

SELECT teacher_id INTO v_teacher_id
FROM disciplines
WHERE discipline_id = p_discipline_id
LIMIT 1;

INSERT INTO session_results (
    student_id,
    discipline_id,
    attempt_number,
    grade,
    status,
    exam_date,
    assessment_type,
    teacher_id
)
VALUES (
    p_student_id,
    p_discipline_id,
    v_new_attempt,
    p_new_grade,
    CASE WHEN p_new_grade >= 60 THEN 'Сдано' ELSE 'Не сдано' END,
    CURRENT_DATE,
    v_assessment_type,
    v_teacher_id
);

```

```

RAISE NOTICE 'Added attempt % for student %, discipline % with grade %',
    v_new_attempt, p_student_id, p_discipline_id, p_new_grade;
END;
$$;

```

Добавим вторую попытку для рассматриваемого студента, пусть он получил 75 баллов.

```
CALL add_exam_retake(1, 5, 75);
```

```

CREATE PROCEDURE
UniversitySession=# CALL add_exam_retake(1, 5, 75);
P-PḡPḡP•PḡPḡPḡPP•: Added attempt 2 for student 1, discipline 5 with grade 75
CALL

```

Как видим, теперь отображаются обе попытки:

```

UniversitySession=# SELECT result_id, student_id, discipline_id, attempt_number, grade, status, assessment_type
UniversitySession=# FROM session_results
UniversitySession=# WHERE student_id = 1 AND discipline_id = 5
UniversitySession=# ORDER BY attempt_number;
result_id student_id discipline_id attempt_number grade status assessment_type
-----
5          1          5          1          40 Не сдано Экзамен
69         1          5          2          75 Сдано Экзамен

```

Создание триггеров

1 Проверка дат начала и окончания обучения студента

Сначала создадим функцию для проверки на то, что дата окончания обучения позже даты начала обучения:

```

CREATE OR REPLACE FUNCTION check_student_dates()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.end_date <= NEW.start_date THEN
        RAISE EXCEPTION 'End date must be after start date';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

```

Далее создаем триггер:

```

CREATE TRIGGER trg_student_dates
BEFORE INSERT OR UPDATE OF start_date, end_date ON students
FOR EACH ROW
EXECUTE FUNCTION check_student_dates();

```

Проверка:

```
UPDATE students SET start_date = '2028-06-30', end_date = '2025-01-09' WHERE student_id = 1;
```

```

UniversitySession=# UPDATE students SET start_date = '2028-06-30', end_date = '2025-01-09' WHERE student_id = 1;
PḡPḡPP'PḡPḡ: End date must be after start date

```

2 Автоматическое обновление статуса сдачи экзамена/зачета по количеству полученных баллов

Посмотрим на запись результата экзамена по дисциплине с id=1 студента с id=1:

```
UniversitySession=# SELECT result_id, grade, status FROM session_results WHERE student_id = 1 AND discipline_id = 1 AND attempt_number = 1;
result_id | grade | status
-----+-----+-----
1 | 13 | Не сдано
```

Код функции для установления статуса “Сдано”, если балл ≥ 60 и “Не сдано” в ином случае:

```
CREATE OR REPLACE FUNCTION update_status_by_grade()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.grade >= 60 THEN
        NEW.status := 'Сдано';
    ELSE
        NEW.status := 'Не сдано';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

Код триггера:

```
CREATE TRIGGER trg_update_status
BEFORE INSERT OR UPDATE OF grade ON session_results
FOR EACH ROW
EXECUTE FUNCTION update_status_by_grade();
```

Попробуем теперь установить для рассматриваемого студента балл 80:

```
UPDATE session_results
SET grade = 80
WHERE student_id = 1 AND discipline_id = 1 AND attempt_number = 1;
```

Статус изменился:

```
UniversitySession=# UPDATE session_results
UniversitySession=# SET grade = 80
UniversitySession=# WHERE student_id = 1 AND discipline_id = 1 AND attempt_number = 1;
UPDATE 1
UniversitySession=# SELECT result_id, grade, status FROM session_results WHERE student_id = 1 AND discipline_id = 1 AND attempt_number = 1;
result_id | grade | status
-----+-----+-----
1 | 80 | Сдано
```

3 Ограничение количества попыток (не более 3)

Сейчас у студента с id=1 есть две попытки сдачи предмета с id=5:

```
UniversitySession=# SELECT attempt_number FROM session_results WHERE student_id = 1 AND discipline_id = 5 ORDER BY attempt_number;
attempt_number
-----
1
2
```

Код функции для проверки на то, что номер попытки не превышает 3:

```
CREATE OR REPLACE FUNCTION check_max_attempts()
RETURNS TRIGGER AS $$
DECLARE
    attempt_count INT;
BEGIN
    SELECT COUNT(*) INTO attempt_count
    FROM session_results
    WHERE student_id = NEW.student_id
    AND discipline_id = NEW.discipline_id;
```

```

    IF attempt_count >= 3 THEN
        RAISE EXCEPTION 'Maximum 3 attempts allowed for student % in discipline %',
NEW.student_id, NEW.discipline_id;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

```

Код триггера:

```

CREATE TRIGGER trg_max_attempts
    BEFORE INSERT ON session_results
    FOR EACH ROW
    EXECUTE FUNCTION check_max_attempts();

```

Вставим 3 попытку:

```

INSERT INTO session_results (student_id, discipline_id, attempt_number, grade, status,
exam_date, assessment_type, teacher_id)
VALUES (1, 5, 3, 50, 'Не сдано', CURRENT_DATE, 'Экзамен', 5);

```

Теперь попробуем вставить 4 попытку:

```

INSERT INTO session_results (student_id, discipline_id, attempt_number, grade, status,
exam_date, assessment_type, teacher_id)
VALUES (1, 5, 4, 62, 'Сдано', CURRENT_DATE, 'Экзамен', 5);

```

```

UniversitySession=# INSERT INTO session_results (student_id, discipline_id, attempt_number, grade, status, exam_date, assessment_type, teacher_id)
pe, teacher_id)
UniversitySession=# VALUES (1, 5, 4, 62, 'Сдано', CURRENT_DATE, 'Экзамен', 5);
PhpPPPP'PpPp: Maximum 3 attempts allowed for student 1 in discipline 5

```

4 Запрет повторной успешной сдачи (нельзя вставить новую попытку, если уже есть сданная)

У студента с id=1 есть запись об успешной сдаче экзамена по предмету с id=1:

```

UniversitySession=# SELECT grade, status FROM session_results WHERE student_id = 1 AND discipline_id = 1 AND grade >= 60;
grade | status
-----+-----
80    | Сдано

```

Код функции для проверки на то, сдан ли уже предмет:

```

CREATE OR REPLACE FUNCTION prevent_duplicate_success()
RETURNS TRIGGER AS $$
DECLARE
    has_success INT;
BEGIN
    SELECT COUNT(*) INTO has_success
    FROM session_results
    WHERE student_id = NEW.student_id
        AND discipline_id = NEW.discipline_id
        AND grade >= 60;

    IF has_success > 0 THEN
        RAISE EXCEPTION 'Discipline already passed';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

```

Код триггера:

```
CREATE TRIGGER trg_prevent_duplicate_success
  BEFORE INSERT ON session_results
  FOR EACH ROW
  EXECUTE FUNCTION prevent_duplicate_success();
```

Попробуем добавить для рассматриваемого студента еще одну попытку успешной сдачи:

```
INSERT INTO session_results (student_id, discipline_id, attempt_number, grade, status,
exam_date, assessment_type, teacher_id)
VALUES (1, 1, 2, 85, 'Сдано', CURRENT_DATE, 'Экзамен', 1);
```

```
UniversitySession=# INSERT INTO session_results (student_id, discipline_id, attempt_number, grade, status, exam_date, assessment_ty
pe, teacher_id)
UniversitySession=# VALUES (1, 1, 2, 85, 'Сдано', CURRENT_DATE, 'Экзамен', 1);
PhpPPPPPP: Discipline already passed
```

5 Запрет удаления группы, в которой есть студенты

Посмотрим, в каких группах сколько студентов:

```
SELECT sg.group_id, sg.group_number, COUNT(s.student_id) AS student_count
FROM student_groups sg
LEFT JOIN students s ON sg.group_id = s.group_id
GROUP BY sg.group_id, sg.group_number
ORDER BY sg.group_id;
```

```
UniversitySession=# SELECT sg.group_id, sg.group_number, COUNT(s.student_id) AS student_count
UniversitySession=# FROM student_groups sg
UniversitySession=# LEFT JOIN students s ON sg.group_id = s.group_id
UniversitySession=# GROUP BY sg.group_id, sg.group_number
UniversitySession=# ORDER BY sg.group_id;
 group_id | group_number | student_count
-----+-----+-----
1 | K3241 | 0
2 | K3240 | 0
3 | K3341 | 20
4 | K3340 | 20
```

Код функции:

```
CREATE OR REPLACE FUNCTION prevent_group_deletion()
RETURNS TRIGGER AS $$
DECLARE
  student_count INT;
BEGIN
  SELECT COUNT(*) INTO student_count
  FROM students
  WHERE group_id = OLD.group_id;

  IF student_count > 0 THEN
    RAISE EXCEPTION 'Cannot delete group with % student(s)', student_count;
  END IF;
  RETURN OLD;
END;
$$ LANGUAGE plpgsql;
```

Код триггера:

```
CREATE TRIGGER trg_prevent_group_deletion
  BEFORE DELETE ON student_groups
  FOR EACH ROW
```

```
EXECUTE FUNCTION prevent_group_deletion();
```

Попробуем удалить группу K3340:

```
DELETE FROM student_groups WHERE group_id = 4;
```

```
UniversitySession=# DELETE FROM student_groups WHERE group_id = 4;
РћРѢРР'РЉРђ: Cannot delete group with 20 student(s)
```

6 Автоматическое заполнение full_hours в таблице disciplines

При вставке или обновлении дисциплины автоматически вычислять full_hours = hours_lectures + hours_practice + hours_labs + hours_seminars.

Посмотрим на имеющиеся данные:

```
SELECT discipline_id, name, hours_lectures, hours_practice, hours_labs, hours_seminars,
full_hours
FROM disciplines
LIMIT 5;
```

```
UniversitySession=# SELECT discipline_id, name, hours_lectures, hours_practice, hours_labs, hours_seminars, full_hours
UniversitySession=# FROM disciplines
UniversitySession=# LIMIT 5;
```

discipline_id	name	hours_lectures	hours_practice	hours_labs	hours_seminars	full_hours
2	Жизненный цикл программного обеспечения	32	16	16	0	64
3	Техники публичных выступлений и презентаций	32	16	16	0	64
4	Механика	32	16	16	0	64
5	Объектно-ориентированное программирование	32	16	16	0	64
6	Теория вероятностей	32	16	16	0	64

Код функции:

```
CREATE OR REPLACE FUNCTION calculate_full_hours()
```

```
RETURNS TRIGGER AS $$
```

```
BEGIN
```

```
    NEW.full_hours := NEW.hours_lectures + NEW.hours_practice + NEW.hours_labs +
NEW.hours_seminars;
```

```
    RETURN NEW;
```

```
END;
```

```
$$ LANGUAGE plpgsql;
```

Код триггера:

```
CREATE TRIGGER trg_calc_full_hours
```

```
    BEFORE INSERT OR UPDATE OF hours_lectures, hours_practice, hours_labs,
hours_seminars ON disciplines
```

```
    FOR EACH ROW
```

```
    EXECUTE FUNCTION calculate_full_hours();
```

Обновим часы у дисциплины с id=2:

```
UPDATE disciplines
```

```
SET hours_lectures = 32, hours_practice = 16, hours_labs = 8, hours_seminars = 4
```

```
WHERE discipline_id = 2;
```

Проверим:

```
SELECT discipline_id, hours_lectures, hours_practice, hours_labs, hours_seminars, full_hours
FROM disciplines
```

```
WHERE discipline_id = 2;
```

```
UniversitySession=# SELECT discipline_id, hours_lectures, hours_practice, hours_labs, hours_seminars, full_hours
UniversitySession=# FROM disciplines
UniversitySession=# WHERE discipline_id = 2;
discipline_id | hours_lectures | hours_practice | hours_labs | hours_seminars | full_hours
-----+-----+-----+-----+-----+-----
2 | 32 | 16 | 8 | 4 | 60
```

7 Запрет удаления дисциплины, если она используется в учебном плане

Посмотрим, какие дисциплины сейчас используются:

```
SELECT d.discipline_id, d.name, COUNT(dic.curriculum_id) AS curriculum_count
FROM disciplines d
LEFT JOIN discipline_in_curriculum dic ON d.discipline_id = dic.discipline_id
GROUP BY d.discipline_id, d.name
HAVING COUNT(dic.curriculum_id) > 0
ORDER BY d.discipline_id
LIMIT 5;
```

```
UniversitySession=# SELECT d.discipline_id, d.name, COUNT(dic.curriculum_id) AS curriculum_count
UniversitySession=# FROM disciplines d
UniversitySession=# LEFT JOIN discipline_in_curriculum dic ON d.discipline_id = dic.discipline_id
UniversitySession=# GROUP BY d.discipline_id, d.name
UniversitySession=# HAVING COUNT(dic.curriculum_id) > 0
UniversitySession=# ORDER BY d.discipline_id
UniversitySession=# LIMIT 5;
```

discipline_id	name	curriculum_count
2	Жизненный цикл программного обеспечения	1
3	Техники публичных выступлений и презентаций	1
4	Механика	1
5	Объектно-ориентированное программирование	1
6	Теория вероятностей	1

Код функции:

```
CREATE OR REPLACE FUNCTION prevent_discipline_deletion()
RETURNS TRIGGER AS $$
DECLARE
    usage_count INT;
BEGIN
    SELECT COUNT(*) INTO usage_count
    FROM discipline_in_curriculum
    WHERE discipline_id = OLD.discipline_id;

    IF usage_count > 0 THEN
        RAISE EXCEPTION 'Cannot delete discipline "%" (id=%) because it is used in %
curriculum(s)',
            OLD.name, OLD.discipline_id, usage_count;
    END IF;
    RETURN OLD;
END;
$$ LANGUAGE plpgsql;
```

Код триггера:

```
CREATE TRIGGER trg_prevent_discipline_deletion
BEFORE DELETE ON disciplines
FOR EACH ROW
EXECUTE FUNCTION prevent_discipline_deletion();
```

Попробуем удалить дисциплину с id=2:

DELETE FROM disciplines WHERE discipline_id = 2;

```
UniversitySession=# DELETE FROM disciplines WHERE discipline_id = 2;  
PhREPP'PaPh: Cannot delete discipline "P-P&P·PSPµPSPSC(PW C†P&PeP» PιCбPspιCбP°PjPjPSPsPiPs PsP±PµCfPιPµC†PµPSP&CU" (id=2) because it is used in 1 curriculum(s)
```

Выводы

В результате выполнения лабораторной работы были разработаны и реализованы три хранимые процедуры: повышение стипендии отличникам, перевод студентов на следующий курс и добавление попыток пересдачи экзаменов. Также созданы семь триггеров, обеспечивающих целостность данных (проверка дат, лимит попыток, запрет удаления занятых объектов), автоматическое обновление статусов оценок и расчёт суммарных часов дисциплин.